SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 10%

QUESTIONS: **Total Marks:50**

Annexure A

DATA INTERPRETATION

Code of Conduct:

I ____A. Midhuna (192521302)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A.Midhuna

Annexure A

DATA INTERPRETATION

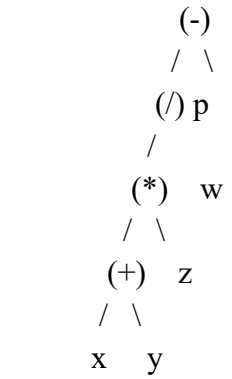
Q1. Intermediate Code Analysis and Optimization
Given the three-address code:
t1 = x + y
t2 = t1 * z
t3 = t2 / w
t4 = t3 - p
Analyze the sequence of operations and construct the equivalent expression tree. Evaluate how temporary variables improve intermediate code generation and reduce repeated computations.

ANSWER:

Intermediate Code Analysis and Optimization
Given three-address code:
t1 = x + y
t2 = t1 * z
t3 = t2 / w
t4 = t3 - p
1. Sequence of Operations
The operations are performed sequentially:

1. Addition: t1 = x + y
2. Multiplication: t2 = t1 * z
3. Division: t3 = t2 / w
4. Subtraction: t4 = t3 - p

Therefore, the equivalent expression is:
[
((x+y) \times z)/w-p
]
2. Equivalent Expression Tree



So, the root of the expression tree is subtraction, followed by division, multiplication, and addition.
3. Role of Temporary Variables

Temporary variables t1, t2, t3, and t4 store intermediate results.

- They break a complex expression into simple operations.
- They make intermediate code generation easier.
- Each temporary result can be reused if needed, avoiding recomputation.
- They help the compiler perform optimization and register allocation.
- For example, (x + y) is calculated once and stored in t1; it does not need to be calculated again if the result is used elsewhere.

Conclusion

Temporary variables provide a clear and systematic representation of complex expressions. They make the intermediate code easier to analyze, optimize, and translate into machine code while helping to reduce repeated computations.

Q2. Symbol Table and Memory Allocation

Given the following symbol table entries:

Variable	Data Type	Size (Bytes)
p	char	1
q	double	8
r	int	4
s	float	4

Interpret how the compiler allocates memory for these variables and calculate the total memory required. Analyze how alignment and data types affect memory organization.

ANSWER:

Given:

Variable	Data Type	Size
p	char	1 byte
q	double	8 bytes
r	int	4 bytes
s	float	4 bytes

1. Memory Allocation

The compiler allocates memory based on the data type and its size.

- p → char → 1 byte
- q → double → 8 bytes
- r → int → 4 bytes
- s → float → 4 bytes

Without considering padding/alignment:

[
1 + 8 + 4 + 4 = \boxed{17\text{ bytes}}
]

So, the minimum total memory required is 17 bytes.

2. Effect of Alignment

Modern systems usually require variables to be stored at suitable memory boundaries.

For example:

- char generally requires 1-byte alignment.
- int generally requires 4-byte alignment.

- float generally requires 4-byte alignment.
- double commonly requires 8-byte alignment.

Therefore, the compiler may insert padding bytes between variables so that each variable starts at an appropriate memory address.
For example, if allocated in the order p, q, r, s:
p → 1 byte
padding → 7 bytes
q → 8 bytes
r → 4 bytes
s → 4 bytes
This can result in approximately:
[
1+7+8+4+4 = \boxed{24\text{ bytes}}
]
depending on the target architecture and compiler's alignment rules.
3. Conclusion
Data types determine the size of variables, while alignment determines their actual memory positions. Padding may increase memory usage from the basic 17 bytes to 24 bytes in a typical 8-byte-aligned layout. Symbol tables help the compiler keep track of each variable's type, size, and memory information during compilation.

Q3. Optimization in Intermediate Code
Given the intermediate code:
t1 = m * n
t2 = p + q
t3 = m * n
t4 = t2 - t3
Trace the execution step-by-step and identify optimization opportunities such as common subexpression elimination and redundant computation removal. Explain how optimization improves execution efficiency.

ANSWER:

Optimization in Intermediate Code
Given the intermediate code:
t1 = m * n
t2 = p + q
t3 = m * n
t4 = t2 - t3
1. Step-by-Step Execution
1. t1 = m * n
   → Calculate m × n and store the result in t1.
2. t2 = p + q
   → Calculate p + q and store the result in t2.
3. t3 = m * n
   → Again calculate m × n and store the result in t3.

4. t4 = t2 - t3
   → Subtract t3 from t2 and store the result in t4.

2. Optimization Opportunity

The expression m * n occurs twice:

t1 = m * n

t3 = m * n

This is a common subexpression. The compiler can calculate it only once and reuse the result.

Optimized code:

t1 = m * n

t2 = p + q

t4 = t2 - t1

Here, t3 is removed because it is redundant.

3. Common Subexpression Elimination

Common Subexpression Elimination (CSE) identifies expressions that have already been calculated and reuses their existing result.

Original:

m * n → calculated twice

Optimized:

m * n → calculated only once

4. Efficiency Improvement

Optimization:

- Reduces the number of multiplication operations.
- Removes unnecessary temporary variable t3.
- Reduces execution time.
- Uses fewer CPU resources and temporary storage.
- Produces more efficient intermediate code.

Conclusion

The original code performs m * n twice unnecessarily. By applying common subexpression elimination and redundant computation removal, the compiler calculates it once and reuses the result, improving the overall execution efficiency and code quality**.**

Q4. Control Flow and Boolean Expression Handling

Given the Boolean expression:

if (x > y || p < q)

Intermediate code:

if x > y goto L1

if p < q goto L1

goto L2

L1: ...

Interpret the control flow of the given intermediate representation and explain how short-circuit evaluation minimizes unnecessary condition checking during execution.

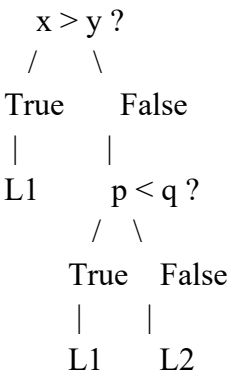
ANSWER:

Given Boolean expression:
if (x > y || p < q)
Intermediate code:
if x > y goto L1
if p < q goto L1
goto L2

L1: ...
1. Control Flow Interpretation
The expression uses the OR (||) operator.

- First, the compiler checks x > y.
- If x > y is true, control immediately jumps to L1.
- If x > y is false, the compiler checks p < q.
- If p < q is true, control jumps to L1.
- If both conditions are false, control goes to L2.

Flow:



2. Short-Circuit Evaluation
Short-circuit evaluation means the second condition is checked only when necessary.
For the OR operation:
x > y || p < q
If x > y is already true, there is no need to evaluate p < q, because the entire expression is definitely true.
For example:
x = 10, y = 5
Since x > y is true, execution directly goes to L1 without checking p < q.
3. Advantages
Short-circuit evaluation:

- Avoids unnecessary condition checking.
- Reduces the number of instructions executed.
- Improves execution speed.
- Can avoid evaluating expensive or unnecessary expressions.
- Produces efficient control-flow code.

Conclusion
The intermediate representation implements the OR condition using conditional jumps. Short-circuit evaluation ensures that p < q is evaluated only when x > y is false, thereby reducing unnecessary computations and improving execution efficiency.

Q5. Backpatching and Flow Control
Given:

Instruction	**Target**
200: if x==y goto _	
201: goto _	

Backpatch lists:

- truelist = {200}
- falselist = {201}

Analyze the role of backpatching in intermediate code generation. Demonstrate how target addresses are updated during control flow handling and explain its importance in Boolean expression translation.

ANSWER:

Given the intermediate code:

Instruction	Target
200: if x == y goto _	Unknown
201: goto _	Unknown

Backpatch lists:

- truelist = {200}
- falselist = {201}

1. Role of Backpatching
Backpatching is a technique used by the compiler to fill in the unknown target addresses of jump instructions after the required target locations become known.
Here, _ represents an address that is not yet known during initial code generation.
2. Backpatching the True List
The instruction at address 200 is:
200: if x == y goto _
Since 200 belongs to truelist, when the true destination is determined as, for example, 202, the compiler updates:
200: if x == y goto 202
3. Backpatching the False List
The instruction at address 201 is:
201: goto _
Since 201 belongs to falselist, if the false destination is 203, it is updated to:
201: goto 203
Thus, the completed code becomes:

200: if x == y goto 202
201: goto 203
202: ...
203: ...
4. Importance in Boolean Expression Translation
Backpatching is especially useful for Boolean expressions containing:

- AND (&&)
- OR (||)
- NOT (!)
- Conditional statements
- if-else statements
- Loops

It allows the compiler to generate jump instructions before knowing their final destinations.
Conclusion
Backpatching connects incomplete jump instructions with their correct target addresses. It makes Boolean expression translation and control-flow generation flexible and efficient, especially when the final locations of statements are not known during the initial intermediate-code generation.

RUBRICS FOR CALCULATION

Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure,	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

		visuals, and terminology			